

PolyForge: Joint Cross-Layer Control of Replicas, Semantic Cache, and Model Tier for Multi-Tenant Hybrid Serving

A thesis submitted in partial fulfilment of the requirements
for the degree of Master of Science

Mohaiminul

[Department]

[University]

Supervisor: [Supervisor]

July 2026

Abstract

Multi-tenant platforms increasingly serve *hybrid* workloads — classic CRUD traffic and AI inference from the same tenants on the same infrastructure. The layers that govern the cost and quality of that serving — replica autoscaling, semantic response caching, and model-tier routing — are today controlled by independent, per-layer policies. This thesis argues, and then measures, that per-layer local optima do not compose into a joint optimum: a controller that plans *replicas, semantic-cache budget, and model tier together*, against one cost-SLO-fairness objective, dominates tuned per-layer baselines.

PolyForge is that controller: a Kubernetes-native control plane with a five-class workload taxonomy, an online classifier, a receding-horizon joint planner (JCAC), and an operator that actuates plans under guardrails. The evaluation is fully pre-registered — seven protocols committed and pushed before their first run — with tuned baselines, paired seeds, effect sizes with bootstrap confidence intervals, and published nulls.

As measured: on a 1,800-run blocked factorial matrix PolyForge beats every tuned baseline (HPA, KEDA, FIRM-replica, GPTCache-LRU, static) on the composite objective and on cost ($|d_z|$ 0.66–1.44); on replayed real demand (BurstGPT, 10.63M requests) the powered confirmatory replay ($n=96$) shows -70% cost against tuned HPA/KEDA/FIRM ($p \leq 4.3 \times 10^{-6}$) with a disclosed $+0.07$ violation trade; on real LMSYS-Chat-1M prompts the adaptive semantic cache reaches 29.6% hit rate at cosine 0.85 ($+93\%$ over a non-strawman fixed baseline) with hit *precision* measured rather than assumed; fairness beats both a FIRM-style and a VTC-style baseline on Jain’s index at lower cost; and a timing side channel in shared semantic caches is demonstrated (AUC 0.88) and closed by per-tenant partitioning at 24% latency cost, dominating padding and TTL-jitter defenses. Two pre-registered SLO hypotheses failed and are published as failures; the honest SLO claim is violation *parity* with tuned reactive scalers at 43–48% lower cost. All headline numbers are decision-quality measurements under a stated system model, calibrated against a real inference server and a real GPU tier bench; live-cluster validation is ordinal-only by design, with the HPA arm verified and the JCAC arm wired behind an executable actuation gate.

Contents

1	Introduction	5
1.1	The problem: hybrid serving is controlled one layer at a time	5
1.2	Thesis statement	5
1.3	What was built	6
1.4	Contributions	6
1.5	Scope and honesty of claims	7
1.6	Document structure	7
2	Background and Related Work	8
2.1	Autoscaling for inference serving	8
2.2	Semantic caching for LLM serving	9
2.3	Multi-tenant fairness	9
2.4	The 2026 serving substrate, and where PolyForge sits	9
2.5	Capability gap summary	10
3	System Design	11
3.1	The control loop, end to end	11
3.2	The JCAC formulation	12
3.3	Construction-level guarantees	13
3.4	Fairness and security in the design	13
3.5	Design decisions with alternatives	14
4	Implementation	15
4.1	Components and code layout	15
4.2	Engineering gates	15
4.3	Deliberate deviations from the roadmap	16
4.4	The live evaluation arm, and two desk-found defects	16
4.5	Scale behavior of the deployed planner	17
5	Experimental Methodology	18
5.1	Pre-registration	18
5.2	Baselines, tuned on our own objective	18
5.3	Substrates and separation rules	18
5.4	Statistics	19
5.5	Metrics	19
5.6	The workload matrix	20

5.7	Honest-nulls policy	20
6	Evaluation	21
6.1	Scoreboard	21
6.2	The headline matrix: 1,800 runs plus 500 ablations	22
6.3	Advanced tier: forecasters, realism, self-calibration	22
6.4	The v2 pre-registered matrix: an SLO hypothesis fails in public	25
6.5	The v3 overload matrix: a failed hypothesis and a confirmed mechanism	25
6.6	Real demand: the BurstGPT replays	26
6.7	Forecasting on real traces: a prediction fails to transfer, a boundary reproduces	26
6.8	The semantic cache on real prompts	27
6.9	Fairness: two dedicated baselines beaten, one own term nulled	27
6.10	Security: a timing side channel and its defense frontier	28
6.11	Live-cluster validation: ordinal by design	28
6.12	Summary: which number to cite	29
7	Discussion	31
7.1	Threats to validity	31
7.2	Construct scope	32
7.3	Limitations	32
7.4	The honest-nulls ledger as a contribution	33
8	Conclusion	34
8.1	What was established	34
8.2	Future work	34
8.3	Closing remark	35
A	Claims Traceability	36

List of Figures

3.1	The PolyForge control loop. Requests (CRUD + AI, JWT-scoped per tenant) hit the control plane and AI gateway; per-request telemetry is aggregated into per-tenant feature windows; the classifier labels each tenant’s workload; every 10 s the JCAC planner emits a joint plan (replicas, cache budget, model tier per tenant); the operator actuates it under guardrails against the same data plane the requests hit.	11
6.1	Cost by system, mean $\pm$ 95% CI over all 300 matrix cells per system (log scale): joint control converts visibility into savings rather than over-provisioning.	22
6.2	The cost-violation plane (means $\pm$ 95% CI): every baseline is dominated on at least one axis without winning the other.	23
6.3	SLO violation by workload class (mean $\pm$ 95% CI): the taxonomy’s control implications made measurable — agentic latency is a tier-knob problem no replica-only controller solves.	23
6.4	One agentic run, blow by blow: joint control in a single trace.	24
6.5	Ablation deltas (red = worse than full PolyForge; * = $p < 0.01$).	24
6.6	Forecast ablation: damped Holt beats the trend default on violation at equal cost; online-seasonal helps only where a period exists.	25
6.7	Left: membership leakage from response time in a shared semantic cache; per-tenant caching pins the attacker at chance. Right: isolation costs hit rate; demand-proportional sizing recovers part of the penalty.	28
6.8	Defense frontier: worst-case membership-inference AUC vs. share of the cache’s latency benefit given up. Per-tenant partitioning dominates the frontier rather than merely joining it.	29

List of Tables

2.1	Substrate mapping of PolyForge’s three knobs onto the 2026 LLM-serving stack (from docs/RELATED_WORK.md §4).	9
4.1	Implementation map.	15
6.1	Verdicts across the six contested segments (RESULTS_MASTER.md).	21
6.2	Disambiguation table (from RESULTS_MASTER.md); the conservative column is the thesis’s claim.	30
A.1	Claim-to-evidence map. Raw run databases (DuckDB) are archived under eval/results/; the Zenodo deposit bundles them with a manifest (docs/RELEASE_CHECKLIST.md).	36
A.1	Claim-to-evidence map. Raw run databases (DuckDB) are archived under eval/results/; the Zenodo deposit bundles them with a manifest (docs/RELEASE_CHECKLIST.md).	37

Chapter 1

Introduction

1.1 The problem: hybrid serving is controlled one layer at a time

Multi-tenant platforms no longer serve one kind of traffic. The same tenants, on the same clusters, issue classic CRUD requests and AI inference requests — chat completions, embeddings, multi-step agent chains — often within a single user-facing transaction. Three control surfaces govern what that serving costs and how it feels:

1. **Replica autoscaling** decides how much capacity each service holds. In production this is the Kubernetes Horizontal Pod Autoscaler (HPA) [15], KEDA [14], or a research controller [2, 5, 22, 25].
2. **Semantic response caching** decides which AI answers are served from an embedding-similarity cache instead of a model call [3, 10, 18, 26].
3. **Model-tier routing** decides which of several differently priced model classes answers each request [24].

Each layer has a mature literature and, in every surveyed system, an independent controller: an autoscaler that has never heard of the cache, a cache that does not know what a replica costs, a router that optimizes one request at a time. Each controller can be locally optimal. This thesis starts from the observation that the three layers share one budget, one latency target, and one set of tenants — and asks whether local optima compose.

1.2 Thesis statement

Per-layer local optima do not compose into a joint optimum. A controller that plans replicas, semantic-cache budget, and model tier together — one receding-horizon optimization per control interval, against a single objective in cost, SLO compliance, and cross-tenant fairness — dominates tuned per-layer baselines on that objective, and does so with per-tenant dollar accounting and budget guardrails that no surveyed system provides.

The claim is tested, not asserted: against five baselines each grid-tuned on the thesis’s own objective, under seven pre-registered protocols whose hypotheses — including the two that failed — were committed and pushed before the first run.

1.3 What was built

PolyForge is a Kubernetes-native control plane implementing the thesis statement end to end (Chapter 3):

- a **control plane** (Go) for tenants, projects, API keys, rate limits, and telemetry, with row-level-security-enforced tenant isolation proven by test;
- an **AI gateway** (Go) with multi-backend chat routing, a semantic cache with cost-aware eviction, vector search, and an agent loop;
- a **workload classifier** mapping 12-feature telemetry windows onto a five-class taxonomy with drift detection;
- the **JCAC planner** (*Joint Cache–Autoscaling Control*): receding-horizon model-predictive control over a move-blocked action lattice, minimizing $J = \alpha \text{cost} + \beta \text{SLO violation} + \gamma(1 - \text{Jain})$, deployed as a Python service that imports the simulator’s controller so offline and online planning are the same code;
- a **Kubernetes operator** (Go) with four CRDs (Tenant, WorkloadProfile, Policy, Budget) that actuates plans under guardrails: replica bounds, one cache level per step, per-tenant budget caps, and last-good-plan fallback.

1.4 Contributions

1. **Joint cross-layer control.** The first surveyed controller to co-optimize replicas, semantic-cache budget, and model tier against one multi-tenant objective (Chapter 3); ablating jointness costs +2884% on the objective’s cost term (Section 6.2).
2. **A measured composition result.** On a 1,800-run blocked factorial with tuned baselines, PolyForge wins the composite objective against every baseline ($|d_z|$ 0.66–1.44) and is the only Pareto-undominated system; on replayed real BurstGPT demand the powered confirmatory replay shows -70% cost ($n=96$, $p \leq 4.3 \times 10^{-6}$) with a disclosed +0.07 violation trade (Section 6.6).
3. **An honest SLO result.** Two pre-registered attempts to beat tuned reactive scalars on raw violation failed and are published as failures; what holds is violation parity at $-43\ldots -48\%$ cost, a confirmed forecasting mechanism (seasonal $<$ trend on violation where real periodicity exists, $p = 6 \times 10^{-4}$), and a declared-exploratory win in the spike regime where reaction is structurally impossible (Section 6.5).
4. **Real-data cache results with a quality denominator.** On real LMSYS-Chat-1M prompts: 29.6% adaptive hit rate at cosine 0.85 (+93% over a 10%-of-working-set fixed cache), dollar-weighted savings, an empirical hit-rate–capacity curve, and — unique

among surveyed cache papers — measured hit *precision* with its stochasticity ceiling (Section 6.8).

5. **Fairness at the control-plane altitude.** Beats a FIRM-style baseline on Jain’s index ($d_z=1.0$) and a VTC-style replica-level transplant *on Jain itself* at -35% cost; the controller’s own fairness term is honestly nulled under injected interference (Section 6.9).
6. **A security frontier for shared semantic caches.** A timing side channel leaking prompt membership at AUC 0.88, and a defense comparison in which per-tenant partitioning restores attacker-to-chance at 24% latency cost, dominating padding and TTL jitter (Section 6.10).
7. **Construction-level theory.** Block-coordinate optimality of the planner’s two-sweep solver and boundedness/contraction of its self-calibration scale, stated at exactly the altitude the code delivers (Section 3.3).
8. **A pre-registered systems methodology.** Seven protocols pushed before their runs, tuned baselines, paired effect sizes with bootstrap CIs, stopping rules, substrate separation, and a published honest-nulls ledger (Chapter 5) — a methodological contribution independent of any performance number.

1.5 Scope and honesty of claims

Every headline number in this thesis is *decision quality under a stated system model*: a discrete-event simulator whose congestion form was calibrated against a real inference server (the fit direction errs *against* PolyForge’s lean postures) and whose model-tier constants were checked against a real GPU latency bench. Real data enters three ways: replayed BurstGPT demand (10.63M requests) and Azure LLM inference 2024 demand (44.1M requests) for the cost and forecasting claims, and real LMSYS-Chat-1M prompts for the cache claims. Live-cluster validation is *ordinal only* by design — absolutes are never compared across substrates — with the HPA arm verified live and the JCAC arm wired behind an executable actuation gate at the time of writing (Section 6.11). What is *not* claimed: absolute dollar or latency numbers transferring to any production fleet, token-level scheduling fairness, or tail behavior beyond p95 on the simulated substrate.

1.6 Document structure

Chapter 2 positions the work against autoscaling, caching, fairness, and the 2026 serving stack. Chapter 3 presents the architecture, the JCAC formulation, and the two propositions. Chapter 4 covers the implementation and its engineering gates. Chapter 5 defines the pre-registered methodology. Chapter 6 reports all campaigns — wins, nulls, and failures. Chapter 7 confronts threats to validity. Chapter 8 concludes. Appendix A maps every claim to its committed evidence.

Chapter 2

Background and Related Work

This chapter positions PolyForge against four bodies of work: autoscaling for inference serving, semantic caching, multi-tenant fairness, and the production LLM-serving stack of 2025–26. One reading rule applies throughout, and it is the same rule the evaluation enforces (Chapter 5): **numbers from other papers are their numbers on their substrate** — real GPU fleets, their traces, their baselines — while PolyForge’s numbers are decision quality on its own matrix and traces. No comparison in this chapter is a head-to-head cell; the comparison is *shape against shape*. Where a published system is reproducible at simulator scale, PolyForge carries it as a tuned in-matrix baseline instead (HPA, KEDA, FIRM-replica, GPTCache-LRU, static, and iso-cost variants).

2.1 Autoscaling for inference serving

The Kubernetes Horizontal Pod Autoscaler [15] and KEDA [14] are the production defaults: reactive controllers on utilization or event-lag signals. FIRM [22] added fine-grained, SLO-oriented resource management for microservices.

The LLM-serving generation is forecast-aware and hierarchical. SageServe [25] reports ~25% GPU-hour savings on 10M production requests using demand forecasting; Chiron [5] reports up to +90% SLO attainment and +70% GPU efficiency via hierarchical autoscaling; Aladdin [2] jointly places and scales for up to −71% serving cost; an SLO-driven Kubernetes autoscaler [29] reports −31% violation duration at −18% cost. Joint horizontal-plus-vertical scaling [30] and SLO-constrained joint allocation [12] widen a single layer’s action space; INFaaS [24] pioneered model-variant selection (“model-less” serving); Faro [8] targets on-premise inference clusters.

Two observations motivate this thesis. First, the closest systems to “joint” — Aladdin (placement+scaling), the horizontal+vertical scalers, INFaaS-style variant selection — each co-optimize *within* one layer. None of them controls a semantic cache, and none carries cross-tenant fairness in its objective. Second, the published wins are typically measured against default or non-SLO-tuned baselines; the bar in this thesis is deliberately higher — every baseline is grid-tuned on the paper’s own objective (Section 5.2).

2.2 Semantic caching for LLM serving

GPTCache [3] established the pattern — embed the prompt, serve a stored response on similarity — reporting 61.6–68.8% hit rates on its own benchmark. SCALM [26] reports +63% hits over GPTCache; InstCache [10] pre-populates predicted instructions and reports 51.34% on LMSYS-Chat-1M; MeanCache [18] moves the cache user-side and federated; Semantic caching adaptation authors [27] study offline-to-online adaptation of similarity thresholds.

In all of these, *the cache is the system*: its size is an input, not a decision variable, and its interaction with capacity and model choice is out of scope. PolyForge treats cache budget as one of three jointly planned knobs, prices its benefit per tenant in dollars, and adds two measurements the cache literature does not publish: hit *precision* against the actually received response on the same real dataset (Section 6.8), and a timing side-channel analysis with a defense frontier (Section 6.10; cf. the prompt-cache audit of Prompt-cache audit authors [21]).

2.3 Multi-tenant fairness

VTC [28] defined service fairness for LLM serving at token granularity inside a continuous-batching engine, with a proven $2\times$ service bound; Equinox [7], D²LPM [6], and Justitia [13] extend it (worst-case service gaps -42% vs. VTC; locality; efficiency). PolyForge’s fairness sits at a different altitude — capacity allocation across tenants at the control-plane level, scored by Jain’s index [11] over per-tenant SLO satisfaction — and the claims are scoped to that altitude (Section 7.2). The evaluation carries a replica-level transplant of VTC’s least-service-first idea as a tuned baseline (`vtc_replica`); beating it on Jain says PolyForge allocates capacity more evenly, not that it schedules tokens more fairly. Token-level fair schedulers are complementary: they would run *inside* each replica pool PolyForge sizes.

2.4 The 2026 serving substrate, and where PolyForge sits

The production stack of 2025–26 — llm-d [17], AIBrix [1], NVIDIA Dynamo [20], and the Kubernetes Gateway API Inference Extension [16] — actuates on signals PolyForge’s system model does not name: queue depth, in-flight requests, KV-cache pressure, prefill/decode pool split. That is an altitude difference, not a contradiction. PolyForge is the *portfolio* controller above that actuation layer, and each knob has a direct target in it:

PolyForge knob	In the system model	2026 actuation target
replicas	per-tenant capacity	decode-pool size / vLLM pod count
cache MB	semantic-cache budget	gateway semantic-cache budget
tier	model class (1:10:100)	model routing (INFaaS-style; AIBrix)

Table 2.1: Substrate mapping of PolyForge’s three knobs onto the 2026 LLM-serving stack (from `docs/RELATED_WORK.md` §4).

Two design decisions keep the mapping honest: the demand signal sits behind an interface (`planner.DemandSource`), so a token-level source (TTFT, queue depth, KV pressure) replaces the request-rate source without touching the planner; and latency claims stay request-level, scoped away from continuous-batching token dynamics. What the 2026 stack does *not* do

— and where PolyForge sits — is decide jointly, across a tenant portfolio, in dollars, with per-tenant budget guardrails.

2.5 Capability gap summary

Across all surveyed systems, none co-optimizes replicas, semantic cache, and model tier against one multi-tenant objective; none accounts per-tenant dollars with budget guardrails; none publishes a security evaluation of its cache layer; and none pre-registers its hypotheses or publishes its nulls. Those four rows — one architectural, one economic, one adversarial, one methodological — are the thesis’s position in the literature. The full capability matrix, including the rows where surveyed systems dominate PolyForge (live GPU fleets, production scale, proven scheduling bounds), is maintained in `docs/RELATED_WORK.md` and summarized honestly in Section 7.3.

Chapter 3

System Design

PolyForge is an adaptive control plane for multi-tenant clusters serving hybrid workloads. This chapter walks the control loop end to end (Figure 3.1), then presents the JCAC formulation and the two construction-level guarantees the deployed solver carries.

3.1 The control loop, end to end

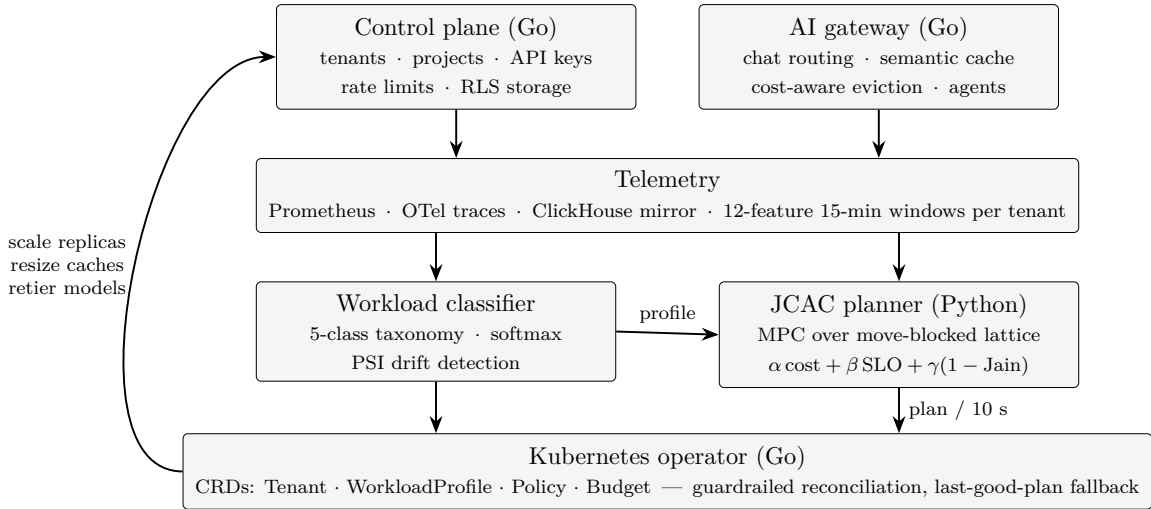


Figure 3.1: The PolyForge control loop. Requests (CRUD + AI, JWT-scoped per tenant) hit the control plane and AI gateway; per-request telemetry is aggregated into per-tenant feature windows; the classifier labels each tenant’s workload; every 10s the JCAC planner emits a joint plan (replicas, cache budget, model tier per tenant); the operator actuates it under guardrails against the same data plane the requests hit.

Control plane. Tenant, project, and API-key management; rate limiting; telemetry ingestion. Tenant isolation is enforced by PostgreSQL row-level security *at the database*, not in the query builder, and is proven by a test that queries the telemetry table with no tenant predicate inside a tenant transaction context and asserts foreign rows are invisible (ADR 0003).

AI gateway. Multi-backend chat routing by plan, length, and budget; a semantic cache with **cost-aware eviction** (evict by expected re-computation dollars, not recency alone); vector search; an agent loop. The cache is per-tenant by default — a security decision, not only a fairness one (Section 6.10).

Telemetry and classification. Every request lands in Prometheus metrics, OpenTelemetry traces, and an analytical mirror. Per tenant, a 15-minute window is reduced to a 12-feature vector (rates, mix, burstiness, latency percentiles, cache affinity); a softmax classifier maps windows onto a five-class taxonomy — steady CRUD, bursty CRUD, cacheable AI chat, uncacheable AI chat, agentic — with population-stability-index drift detection. Each class carries a control implication (e.g., agentic latency needs the tier knob, not more replicas), which the evaluation makes measurable (Figure 6.3).

JCAC planner. A receding-horizon model-predictive controller over a *move-blocked action lattice* (Section 3.2). The deployed Python service imports the simulator’s controller module — offline evaluation and online planning are provably the same code, which is what licenses reading simulator decision quality as deployed decision quality (Section 7.1).

Operator. Four CRDs — Tenant, WorkloadProfile, Policy, Budget — reconciled with guardrails: replica bounds, at most one committed cache level move per step, per-tenant budget caps, ordered finalizer teardown, and last-good-plan fallback when the planner is unreachable or over deadline. When several Policies resolve to one shared target, their clamped contributions are *summed* (pool-tenant semantics) rather than last-writer-wins — a correctness property found and fixed at the desk before the live campaign (Section 4.4).

3.2 The JCAC formulation

At each control step the planner holds tenants $T = \{1, \dots, m\}$. Tenant i ’s decision variable is $x_i = (r_i, c_i, \tau_i)$ — replicas, cache level, model tier — drawn from a finite per-tenant lattice X_i : replica deltas $\Delta r \in \{-2, \dots, 2\}$, cache moves at most one committed level, tier $\tau_i \in \{\text{none, small, mid, large}\}$; hence $|X_i| \leq 60$ and the joint space $X = \prod_i X_i$ is finite. The per-step objective is

$$J(x) = \alpha \frac{1}{S} \sum_i \text{cost}_i(x_i) + \beta \sum_i \text{obj}_i(x_i) + \gamma \sum_i w_i (1 - \text{Jain}(s(x))) + \gamma \sum_i \nu_i \rho_i(x_i) + \lambda \sum_i \text{sw}_i(x_i), \quad (3.1)$$

where $s(x)_i = 1 - \text{viol}_i(x_i)$ is the per-tenant SLO-satisfaction vector entering Jain’s index [11], ν_i is an exogenous interference score from the noisy-neighbor detector, and sw_i counts switches (hysteresis). Feasibility intersects per-tenant budget caps with two cluster-level caps, $\sum_i c_i \leq C$ and $\sum_i r_i \leq R$, both separable in a scan.

Demand enters through a pluggable forecaster. The code default is a linear trend; damped Holt smoothing [9] is the documented recommended default after the in-loop ablation (Section 6.7), and an online-seasonal variant exists for demand with a detected period. The

evaluation shows the right reading is *per-stream*: no fixed method dominates across three real traces; the winner tracks measured periodicity.

The solver is not a MIP: it runs two fixed Gauss–Seidel sweeps over tenants, each block update exactly minimizing J over the ≤ 60 candidates of one tenant with the others held fixed (ADR 0014 records why enumeration beats a solver dependency at this lattice size).

3.3 Construction-level guarantees

Both propositions hold *by construction of the deployed solver* (`research/jcac_sim/controller.py`); neither is asymptotic or probabilistic. Full proofs are committed in `research/analysis/THEORY_V2.md`; the statements and proof shapes follow.

Proposition 1 (Block-coordinate optimality). *Let $x^{(0)}$ be the incumbent configuration and x^* the result of the solver’s two fixed sweeps, each replacing $x_i \leftarrow \arg \min_{x_i \in X_i(x_{-i})} J_i(x_i; x_{-i})$ over the feasible slice. Then x^* is a block-coordinate minimum of J : no single tenant can lower the global objective by unilaterally changing its own configuration.*

Proof shape. Each block update is an exact minimization over a finite set containing the incumbent, so J is non-increasing across updates. A tenant’s best response depends on the others only through sums and the satisfaction multiset, and the feasible slice depends only on residual-cap sums, which are order-independent; a backward-propagation argument over the last moved tenant in sweep two shows every tenant ends at a best response to the final configuration. $\square$

Scope. A block-coordinate minimum is *not* guaranteed to be the global minimizer: the Jain coupling is non-separable, and a simultaneous multi-tenant move could in principle beat a coordinate-wise optimum. The proposition claims exactly what the solver delivers — unilateral-move optimality — and no more.

Proposition 2 (Self-calibration is bounded and contracts on agreement). *Let $g_i^{(t)}$ be tenant i ’s capacity scale under the feedback update with learning rate $\eta = 0.15$, recovery step $\delta = 0.02$, and tolerance $\epsilon = 0.02$. Then (a) $[0.5, 1]$ is forward-invariant and absorbing; (b) the scale can only lower effective capacity — planned load is never below observed load; (c) under sustained agreement between projection and realization, $g_i \uparrow 1$ in at most $\lceil 0.5/0.02 \rceil = 25$ steps and stays there.*

Reading. The mechanism is safe to ship on by default: it can only make the planner more conservative, it is confined to a bounded corrective band by explicit max / min projections (safety does not depend on tuning), and it provably relaxes back to trusting the nominal model rather than ratcheting pessimism forever. Its empirical value is measured under reconfiguration realism ($d_z = +0.46$, Section 6.3).

3.4 Fairness and security in the design

Fairness. The noisy-neighbor detector scores tenants *peer-relatively* (ADR 0015) on eBPF-shaped signals; the score enters the objective as the $\nu_i \rho_i$ penalty. Jain’s index over SLO satisfaction is exported every cycle. The γ -weighted Jain term is a design feature whose measured contribution under injected interference is a published null (Section 6.9); it remains configurable, and the claims rest on the measured outcomes, not on the term’s presence.

Security. Sharing a semantic cache across tenants creates a timing side channel: a hit on a prompt another tenant inserted returns faster, leaking membership. The design defense is per-tenant cache partitioning with demand-proportional sizing — the evaluation quantifies both the attack and the defense frontier (Section 6.10).

3.5 Design decisions with alternatives

Every architectural choice with a live alternative is recorded as an ADR (`docs/adr/0001-0015`). Load-bearing examples: row-level security at the database, not the query builder (0003); an outbox-audited tenant isolation ladder (0008); enumeration over a MIP backend for the planner (0014); peer-relative rather than absolute noisy-neighbor detection (0015). Chapter 4 records the tool-level deviations from the original roadmap and why their outcomes are equivalent.

Chapter 4

Implementation

PolyForge is a working system, not a simulator with a paper wrapped around it. This chapter records what was built, the engineering gates it is held to, the deliberate deviations from the original roadmap, and the wiring that makes the live evaluation arm honest.

4.1 Components and code layout

Component	Code and role
Control plane	<code>cmd/control-plane</code> , <code>internal/</code> : tenant / project / API-key APIs, PostgreSQL with row-level security (SQLite dev path), rate limiting, telemetry ingestion, canary and isolation-ladder admin APIs.
AI gateway	<code>cmd/ai-gateway</code> , <code>internal/ai</code> : multi-backend chat routing (plan/length/budget), semantic cache with cost-aware eviction, vector search, agent loop.
Classifier	<code>internal/classifier</code> , <code>cmd/classifier-train</code> : 12-feature windows → 5-class taxonomy, online labels every 10s, PSI drift detection.
JCAC planner	<code>services/planner</code> + <code>research/jcac_sim</code> : the service imports the simulator’s controller, so offline and online planning are the same code.
Operator	<code>cmd/operator</code> , <code>deploy/helm/polyforge-operator</code> : four CRDs, guardrailed reconciliation, ordered finalizer teardown, OTel-traced reconciles.
Evaluation	<code>eval/</code> : YAML-driven harness, five tuned baselines, DuckDB results, validation and spot-check tooling; kind-cluster live backend.

Table 4.1: Implementation map.

4.2 Engineering gates

Every session’s work passes the same battery before it is committed: `gofmt` clean, `go vet`, `go build`, `go test ./... -count=1` across 26 Go packages, the Python suites (harness, planner, analysis — 41 tests at the time of writing), and OpenAPI linting. CI additionally runs the race detector (`-race -covermode=atomic`) — which cannot run on the development machine — a coverage gate, `golangci-lint`, and the PostgreSQL row-level security suite against a live PostgreSQL service. Two verification norms follow from that split and are kept throughout the project record: tests that only CI can execute are reported as *unproven until a green CI run*, and the session log (`docs/SESSION_LOG.md`) records how every change was verified.

Security-relevant details received the same treatment as research claims. API-key identifiers are drawn independently of the secret (an audit found the ID leaked 64 of 192 entropy bits); keys are hashed with SHA-256 rather than argon2id — sound because the secrets are 192-bit random strings where brute force is infeasible, a deliberate deviation recorded as an ADR precisely because it is defense bait; a panic-recovery middleware returns structured 500s without leaking stack details; the authentication packages are held at $\sim 95\%$ -plus coverage.

4.3 Deliberate deviations from the roadmap

The original roadmap named specific tools; the implementation deviates in three places with equivalent outcomes, each recorded rather than silently “fixed”: the chi router is replaced by Go 1.22+ stdlib `http.ServeMux` method patterns; sqlc and golang-migrate are replaced by hand-written SQL behind repository interfaces plus an idempotent migrator (type-safe access and versioned schema still hold, RLS proven by test); argon2id by SHA-256 as above. The evaluation deviates from the roadmap’s p99 in reporting p95 (Section 5.5).

4.4 The live evaluation arm, and two desk-found defects

The live campaign (Section 6.11) runs on a kind cluster: harness-built images, a k6 load generator, real HPA for the baseline arm, and the operator + planner for the JCAC arm. Wiring the JCAC arm surfaced two defects at the desk that would each have silently invalidated the cluster session:

1. **Demand plumbing that could never have authenticated.** The operator’s feature-demand source sent its token as a bearer, which the control plane treats strictly as a JWT; every read would have been a 401, and the JCAC loop *degrades by design* into silent fallback — the arm would have run “live” while planning nothing. The features endpoint (that endpoint only) now accepts the platform admin key, unit-tested in both directions (the admin key reads any tenant’s features; it still cannot touch other tenant routes).
2. **Last-writer-wins actuation on a shared target.** All eight eval tenants’ Policies name one shared Deployment; each reconcile overwrote the others’ replica counts, which would have pinned the target at a single tenant’s share — JCAC crippled by wiring and mislabeled as JCAC. Policies resolving to the same target now sum their clamped contributions, with deletion re-enqueuing siblings; unit-tested.

Both fixes carry a broader honesty mechanism: the harness ends operator installation with `kubect1 wait --for=condition=Applied` on every Policy, so a run in which the operator never actuated *fails before any load is generated* — the “the controller was actually in the loop” gate is executable, not a promise. Capacity parity is a cell property under test: every arm starts at the same replica count and is ceilinged by the same cluster limits, so no arm can win by being handed more cluster.

4.5 Scale behavior of the deployed planner

The deployed `PlannerCore.plan` path was driven from 8 to 256 tenants on one laptop core (`research/analysis/PLANNER_SCALING.md`): fitted growth exponent **1.45** — the joint fairness term couples tenants, and super-linearity is the measured price of jointness — with p95 cycle time first exceeding the operator’s 3s request timeout at 128 tenants and the 10s control period at 256. Past the timeout the failure mode is the designed one: the operator holds the last good plan and reports `fallback` status. Scaling levers past the crossover (planning cells, incremental fairness partial sums, a faster inner loop) are named with the measurement. Coordination costs outside the planner — CR write fan-out, telemetry aggregation — are not covered by this microbenchmark.

Chapter 5

Experimental Methodology

The evaluation’s credibility rests on rules that were fixed before results existed. This chapter states them; Chapter 6 reports what happened under them — including the hypotheses that failed.

5.1 Pre-registration

Every confirmatory campaign’s protocol — hypotheses, sole treatment, analysis script, and stopping rule — was committed and pushed to the public repository *before its first run*. Seven pre-registrations exist: `PREREG_V2.md`, `PREREG_V3.md`, `PREREG_TRACE.md`, `PREREG_TRACE2.md`, `PREREG_VTC.md`, plus the v1 gate and the Phase-4 acceptance frozen in the roadmap documents. Protocol changes forced by feasibility (e.g. the LMSYS similarity matrix that would not fit in 40 TB) were committed as *declared pre-run amendments* before the gated data was downloaded — the amendment chain is the pre-registration’s honest continuation, visible in git history. Closed campaigns are immutable: stopping rules forbid re-running, widening, or post-hoc tuning, and campaign result files are never edited after their run.

5.2 Baselines, tuned on our own objective

Five baselines: HPA-style reactive scaling, KEDA-style event-driven scaling, a FIRM-style replica controller [22], GPTCache-style fixed semantic caching with LRU [3], and static provisioning; later campaigns add a VTC-style least-service-first replica divider [28] and iso-cost variants. Every baseline is grid-search tuned *on the composite objective this thesis is graded on*, with sweeps committed (`eval/baselines/TUNING.md`, grids in `eval/baselines/grids/`). Grids are bounded at a vendor-sane envelope because J improves monotonically toward over-provisioning — an unbounded grid would tune every baseline into a cost blowout and flatter PolyForge. Iso-cost variants exist to remove the “it just spends less” objection, and the gate they impose is reported FAIL-honest where it fails.

5.3 Substrates and separation rules

Three substrates carry the evidence, and they are never mixed in one table:

1. **The simulator** (`research/jcac_sim`): discrete-event decision-quality substrate. Its congestion form was calibrated against a real inference server — the fitted exponent ($a=0.86$ vs. assumed 1.0) means the model over-penalizes high utilization, i.e. errs *against* PolyForge’s lean postures — and its tier constants were checked against a real GPU bench (measured latency ratios 1:1.5:16.6; same ordering as the priced 1:10:100, compressed scale, disclosed).
2. **Real data replayed through the simulator**: BurstGPT v2.0 [4] demand (10,632,194 requests, 335 days), Azure LLM inference 2024 [19] demand (44.1M requests), and LMSYS-Chat-1M prompts [32] under a real MiniLM encoder [23, 31].
3. **A live kind cluster**: real scheduler, real HPA, real load. Claims from this substrate are *ordinal only* — does the simulator’s ranking reproduce? — never absolute comparisons.

Replay dollars are model-scale; the claim is always the ranking. Samples from different replays are never pooled ($n=16$ and $n=96$ are separate experiments with separate pre-registrations).

5.4 Statistics

Runs are paired by seed: every system sees identical demand realizations, including arrival jitter drawn once per window and shared across systems. Significance is Wilcoxon/paired tests at the pre-registered gate; but at hundreds of paired seeded simulator runs, vanishing p-values measure the simulator’s determinism, not the effect. The *citable unit* is therefore the paired effect size d_z with a 95% bootstrap confidence interval (`EFFECT_SIZES.md` restates the real-trace campaigns this way; all $n=96$ confirmatory intervals exclude zero, and the +0.07 violation trade carries its own CI). P-values appear once, as prereg gate outcomes, and are never headlined below $\sim 10^{-6}$. A declared-before-run weight-sensitivity sweep perturbs the objective weights across 425 cells spanning all campaigns: the win direction never flips, and 416/425 cells keep $p < 0.01$ (`SENSITIVITY_J.md`).

5.5 Metrics

Cost (USD: replica-hours at \$0.048/replica-hr; model-tier calls at market API ratios 1:10:100), SLO violation rate, semantic-cache hit rate, Jain’s index over per-tenant SLO satisfaction, and the composite

$$J = \text{cost} + 2 \cdot \text{violation} + 0.5 \cdot (1 - \text{Jain}), \quad (5.1)$$

normalized per tenant-step — the objective every baseline was tuned on. Latency is reported at **p95, not p99**: the simulator’s latency estimator is a queueing approximation whose tail beyond p95 is not credible, and pretending otherwise would be false precision. This is a documented deviation; the live path measures wall-time latency and can report any percentile.

5.6 The workload matrix

The headline matrix is a blocked factorial: 5 workload classes (from the taxonomy) $\times$ 4 tenant-mix profiles $\times$ 3 cluster sizes $\times$ 6 systems $\times$ 5 repetitions = 1,800 runs, plus 500 ablation runs. Eight tenants per cluster is the studied factor (composition, not population); the planner’s tenant-count scaling is measured separately (Section 4.5). The v3 overload cells were sized by *arithmetic on committed model constants* — burst climbs exceeding the shared ± 2 -replicas-per-interval actuation clamp — pinned by unit tests, not tuned by trial runs, with a same-envelope `ramp_gentle` control cell where the hypothesized advantage should not (and does not) appear.

5.7 Honest-nulls policy

Negative and failed results are published with the same prominence as wins, in a dedicated ledger (`RESULTS_MASTER.md`): the two confirmatory SLO failures (v2 H1, v3 H1’), the under-powered first replay (HT), the fairness-term null (γ), the seasonal-forecasting non-transfer from synthetic to real demand, the structurally impossible raw per-metric gate, and the cache hit-quality result that most $\tau=0.85$ hits fail the response-agreement proxy. Goalpost-moving is invisible when it happens in private; this project’s position is that it happened in public, in the git history, where an examiner can diff it.

Chapter 6

Evaluation

This chapter reports every campaign in the order it was run, under the rules of Chapter 5: wins, nulls, and failures alike. Every number traces to a committed, machine-generated measurement record (Appendix A); where a summary here and a campaign file could disagree, the campaign file wins.

6.1 Scoreboard

Segment	Verdict	The citable claim
Cost	Won (confirmatory; synthetic + real)	Beats every tuned baseline on cost in the 1,800-run matrix (d_z 0.66–1.12); on real BurstGPT demand: -70% cost vs. tuned HPA/KEDA/FIRM ($p \leq 4.3 \times 10^{-6}$, $n=96$).
SLO	Closed honestly	Violation parity with tuned HPA/KEDA at $-43\ldots-48\%$ cost; beats FIRM. Overload: reactive scalers buy attainment at $3.2\text{--}3.4\times$ spend (H1' FAIL); forecast mechanism confirmed (H2' PASS); spike-class exploratory win.
Fairness	Won (two dedicated baselines)	Beats FIRM on Jain ($d_z=1.0$); beats tuned VTC-replica on Jain itself (0.9705 vs. 0.9599) at -35% cost. Own γ -term honestly nulled.
Forecasting	Won (on real data)	Damped Holt -26.9% RMSE on real BurstGPT; the synthetic seasonal prediction did not transfer and is published as such; on Azure LLM 2024 the mechanism boundary reproduces — the winner tracks measured periodicity.
Cache	Closed (real-data headline)	29.6% hit at cosine 0.85 / 48.8% at 0.70 on real LMSYS-Chat-1M; $+93\%$ vs. 10%-fixed; \$0.296/1k queries; hit precision measured with its stochasticity ceiling.
Security	Won (uncontested novelty)	Timing side channel at AUC 0.88; per-tenant partitioning restores chance at 24% latency cost, dominating padding and TTL jitter.

Table 6.1: Verdicts across the six contested segments (RESULTS_MASTER.md).

On the composite objective J — the objective every baseline was tuned on — PolyForge wins against *every* baseline in *every* campaign that tested it: v1 ($p \leq 4.6 \times 10^{-25}$), v2 ($p \leq 6.7 \times 10^{-25}$), v3 overload ($p \leq 1.3 \times 10^{-11}$), the VTC slice ($p = 3.3 \times 10^{-19}$), and the real-demand replay ($p \leq 5.5 \times 10^{-5}$) — and is the only Pareto-undominated system in the matrix (Figure 6.2).

6.2 The headline matrix: 1,800 runs plus 500 ablations

The blocked factorial of Section 5.6 with all baselines grid-tuned on J . PolyForge beats every tuned baseline on composite J ($p \leq 4.6 \times 10^{-25}$, $|d_z|$ 0.66–1.44) and on cost against all five (Figure 6.1); it holds violations near the over-provisioned floor at a fraction of that floor’s cost, and sits alone on the cost–violation frontier (Figure 6.2).

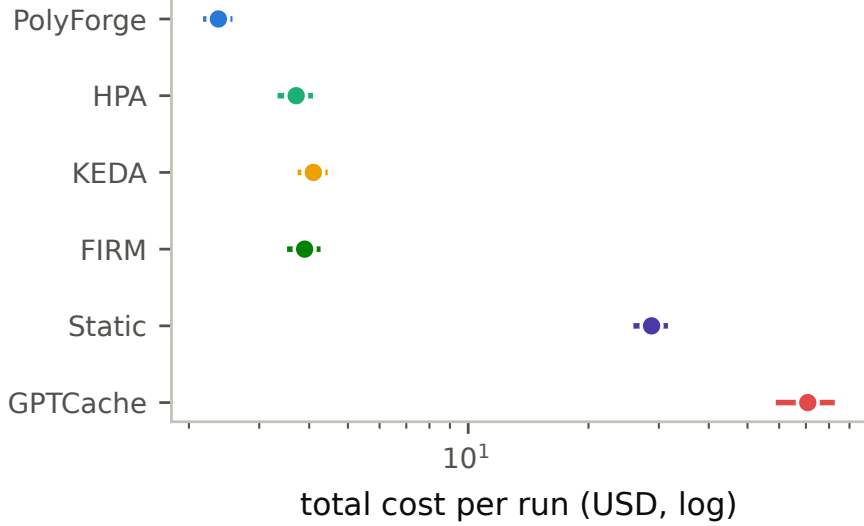


Figure 6.1: Cost by system, mean $\pm$ 95% CI over all 300 matrix cells per system (log scale): joint control converts visibility into savings rather than over-provisioning.

One pre-committed gate **fails as measured** and is reported: the roadmap’s raw per-metric gate (win every metric outright) is structurally impossible against by-construction winners — over-provisioned Static and cache-maxed GPTCache take the SLO and hit-rate columns at 12–30 $\times$ PolyForge’s spend. The iso-cost variants exist for exactly this comparison (Section 6.4).

Where baselines break is class-structured, as the taxonomy’s control-implication column predicts: replica-only controllers cannot fix agentic latency, because it needs the tier knob (Figure 6.3). The single-trace view shows the composed behavior directly: within the first minute of an agentic run, PolyForge fills its replica budget *and* quadruples the semantic cache, while HPA chases each burst with the only knob it has (Figure 6.4).

Ablations (500 runs). Removing joint control costs +2884% on the cost metric; removing cost-aware eviction +35.7%; removing the classifier +2.1% cost and -45% relative hit rate; each of the four components carries independent, statistically significant weight (Figure 6.5). The composition claim of Section 1.5 is this measurement.

6.3 Advanced tier: forecasters, realism, self-calibration

The in-loop forecast ablation found headroom in the system’s own default: the linear-trend forecaster overreacts to bucket noise, and damped Holt smoothing beats it by -16.3% SLO violation at equal cost ($p = 0.0027$; Figure 6.6). Holt became the documented recommended

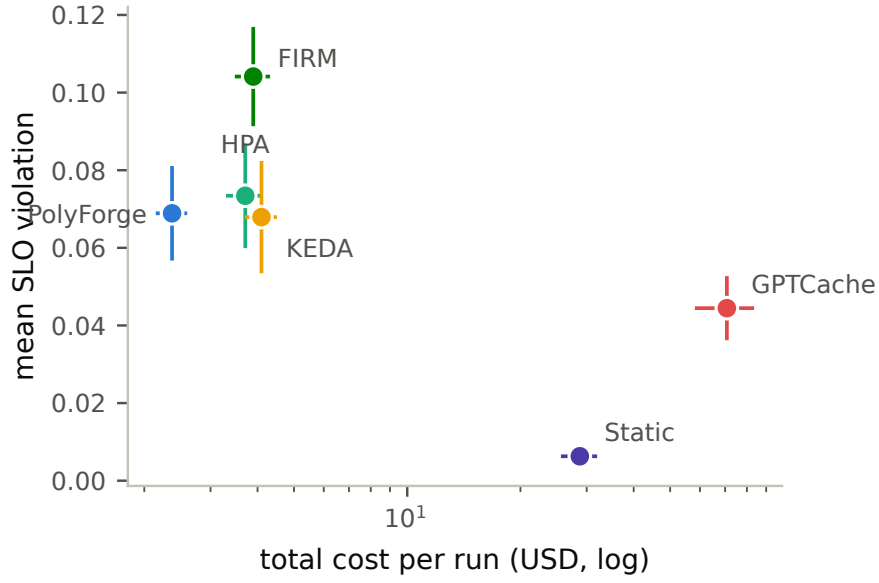


Figure 6.2: The cost-violation plane (means $\pm$ 95% CI): every baseline is dominated on at least one axis without winning the other.

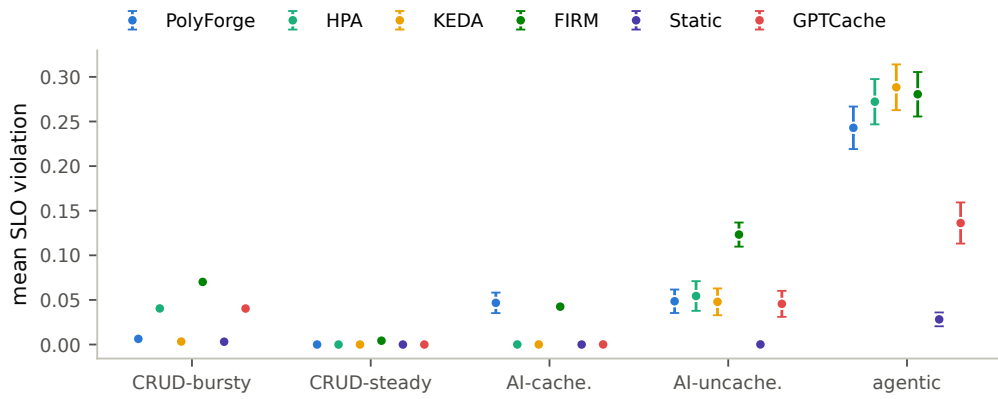


Figure 6.3: SLO violation by workload class (mean $\pm$ 95% CI): the taxonomy’s control implications made measurable — agentic latency is a tier-knob problem no replica-only controller solves.

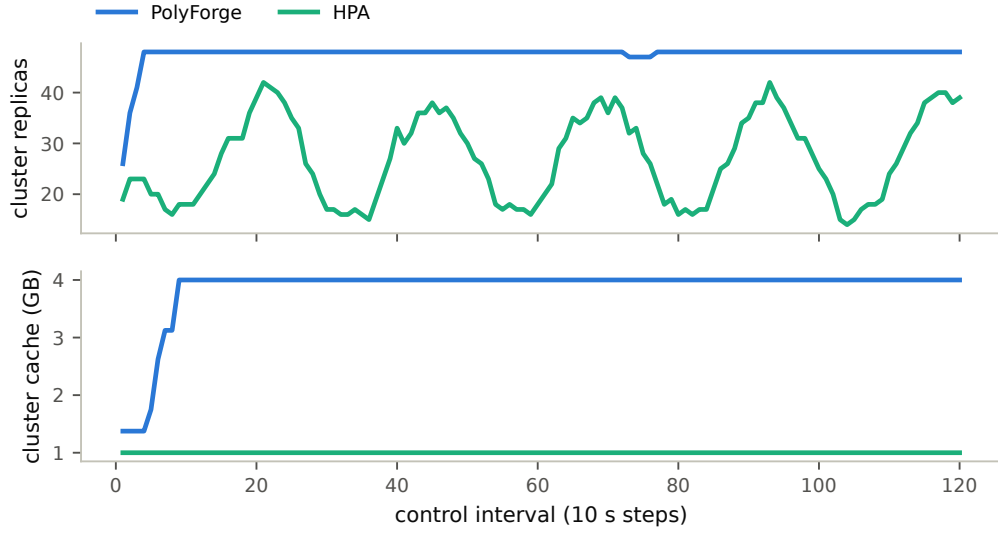


Figure 6.4: One agentic run, blow by blow: joint control in a single trace.

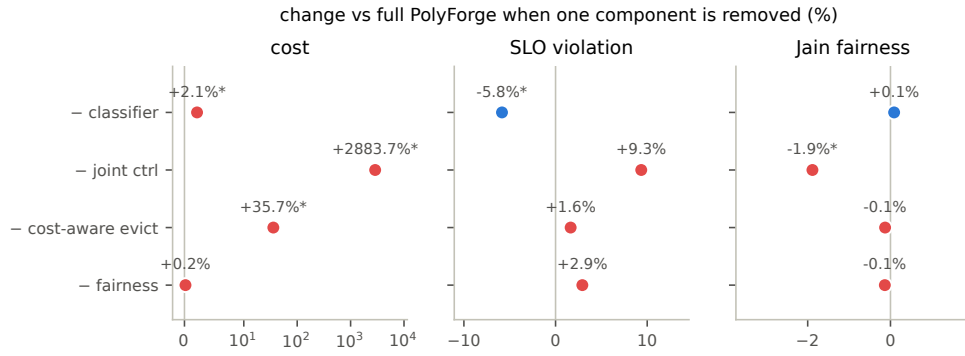


Figure 6.5: Ablation deltas (red = worse than full PolyForge; * = $p < 0.01$).

default; trend stays the code default so the committed 1,800-run headline remains bit-reproducible. Under reconfiguration realism (replica startup lag and half-warm caches, billed immediately), reactive scalers pay for capacity that misses the burst it was bought for, while PolyForge’s hysteresis holds the frontier; self-calibration (Proposition 2) earns $d_z=+0.46$ ($p = 0.029$), concentrated on the bursty classes where trusting nominal capacity hurts most.

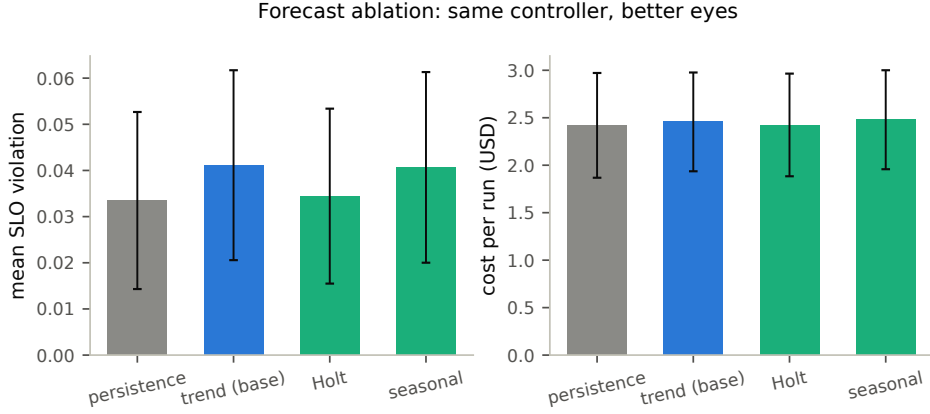


Figure 6.6: Forecast ablation: damped Holt beats the trend default on violation at equal cost; online-seasonal helps only where a period exists.

6.4 The v2 pre-registered matrix: an SLO hypothesis fails in public

PREREG_V2.md froze `jcac_v2` (Holt + adaptive capacity) and two hypotheses before any of the 2,100 runs. **H1 FAILED:** SLO violation is not significantly below tuned HPA/KEDA — what holds is violation *parity* at $-43 \dots -48\%$ cost, and a clear win over FIRM ($p = 2 \times 10^{-19}$). **H2 PASSED:** no large-effect self-regression, and cost improves -11% over v1 ($p = 4 \times 10^{-36}$). The iso-cost gate reshapes the by-construction winners of Section 6.2: budget-matching flips Static from win-by-spending to a 3-of-5 loss, and 36 of 60 cells are budget-infeasible for the cache-max posture (it outspends the budget $36\times$). Per the pre-registered stopping rule, `jcac_v2` was not re-tuned after the H1 failure; v1 `jcac` remains the headline system.

6.5 The v3 overload matrix: a failed hypothesis and a confirmed mechanism

The v2 matrix’s demand shapes are smooth enough that reactive scalers keep up; when everyone can react in time, everyone ties on violation and the only degree of freedom left is cost. PREREG_V3.md therefore constructed overload cells where demand climbs faster than the shared ± 2 -replicas-per-interval actuation clamp — sized by arithmetic on committed model constants, pinned by unit tests (Section 5.6) — with `jcac_seasonal` named in advance as the sole treatment. 1,200 runs, all valid.

H1’ FAILED, and the failure is informative: tuned HPA/KEDA buy overload attainment with $3.2\text{--}3.4\times$ PolyForge’s spend (seasonal is $-69/-71\%$ on cost at $+0.05/+0.09$ violation), and only PolyForge honors the cluster capacity cap at these amplitudes (a

disclosed harness asymmetry, strict against us). **H2' PASSED**: seasonal forecasting reduces violations against trend ($p = 6 \times 10^{-4}$) at no cost regression — the mechanism is real. In the declared-exploratory spike class, where reaction is structurally impossible, the full predicted phenomenon appears: seasonal beats HPA on violation ($p = 5 \times 10^{-5}$, $d_z = 0.57$) at -46% cost. Composite J : wins against all three baselines ($p \leq 1.3 \times 10^{-11}$). The pre-registered stopping rule closes confirmatory SLO attempts for this thesis: two attempts, two published failures, one confirmed mechanism.

6.6 Real demand: the BurstGPT replays

Real BurstGPT v2.0 demand [4] — 10,632,194 requests over 335 days, normalized deterministically — replayed through the committed system model in 6-hour windows, demand scale frozen pre-run, arrival jitter drawn once per window and shared across systems.

First sample ($n=16$): HT FAILED at the $p < 0.01$ gate — underpowered ($p \approx 0.07\text{--}0.11$) — but every point estimate reproduces the v1 ranking. The sample stands as measured; it is not the citable headline. **Powered second sample (PREREG_TRACE2.md, $n=96$ independent seeds, $\sim 94\%$ power): HT2 PASSED with disclosure** — J beats HPA/KEDA/FIRM at $p \leq 5.5 \times 10^{-5}$ ($d_z -0.43 \dots -0.47$), **cost** -70% ($-\$77 \dots -\80 per window, $p \leq 4.3 \times 10^{-6}$), and the pre-registered disclosure is honored: a violation trade of $+0.07$ ($p < 10^{-6}$) rides along — the same attainment-for-multiples-of-spend trade quantified everywhere else. Both collection segments show the identical effect; samples are never pooled. Effect sizes with bootstrap CIs, the citable unit, are tabulated in EFFECT_SIZES.md; all $n=96$ confirmatory intervals exclude zero.

6.7 Forecasting on real traces: a prediction fails to transfer, a boundary reproduces

The synthetic stand-in (textbook diurnal, autocorrelation 0.85 at 24 h) predicted seasonal forecasting would dominate: -44.8% RMSE. On the real BurstGPT trace the periodicity is weak (best autocorrelation ≤ 0.49 at 8 h) and **the prediction does not transfer**: seasonal never beats trend; damped Holt wins segment 1 (-26.9% RMSE), persistence wins segment 2 (-16.6%), and trend is the worst choice on real data. The recommendation followed the data (Holt is jcac_v2's forecaster).

The second real trace — Azure LLM inference 2024 [19], 44.1M requests, two production workloads — reproduces the *mechanism boundary* through the identical frozen decomposition: seasonal wins exactly on the stream with a genuine daily cycle (code: -15.6% RMSE at autocorrelation 0.57, lag 24), loses on the weakly periodic conversation stream ($+48.5\%$; persistence wins), and pooling the streams destroys forecastability (seasonal $+104\%$). Across three real streams no fixed forecaster dominates; the winner tracks measured periodicity. That is direct external support for *per-tenant* forecasting and the pluggable-forecaster design — and it is the scoped version of the v3 H2' result.

6.8 The semantic cache on real prompts

On LMSYS-Chat-1M [32] (seeded reservoir sample of 200,000 from 2,015,645 user turns; MiniLM embeddings; block-wise exact nearest neighbor, verified bit-identical against the dense computation; feasibility amendments committed before the gated download):

- **Hit rate:** adaptive 29.6% at cosine 0.85, 48.8% at 0.70. InstCache’s 51.34% on the same dataset [10] is a different mechanism (pre-populated predicted instructions) and is placed beside, never head-to-head.
- **Against a non-strawman fixed baseline:** +93% over a fixed cache at 10% of the inserted working set (29.6% vs. 15.3%). In-matrix, adaptive sizing gives +83% over fixed (0.106 vs. 0.058).
- **Dollars:** \$0.296 saved per 1,000 queries at mid-tier pricing — the metric cache-only papers cannot produce, because it requires the tenant-tier accounting.
- **Capacity curve:** empirical $h(K)$ fit — $h_{\max} = 0.285$, $K_{1/2} \approx 6,569$ entries (≈ 19 MB). The simulator’s assumed curve was optimistic; the fit is documented beside it, and the committed constants are unchanged (bit-reproducibility; adopting them requires a new pre-registration).

Hit quality. A hit that serves the wrong answer is not a saving, so precision was measured under a frozen protocol (CACHE_PRECISION.md): the probability, given a hit, that the served response agrees with the actually received response, at agreement threshold $\rho = 0.70$, with same-model and first-turn strata declared pre-run. At $\tau=0.85$: precision 0.313 overall, 0.443 same-model, 0.353 first-turn; 165.5 incorrect hits per 1,000 queries reported next to the savings. The built-in calibration bounds the reading: near-duplicate prompts ($\tau \geq 0.95$) agree only 33.8% — 25 models and sampling temperature dominate the absolute level. The citable claims are therefore the *relative* readings: precision rises monotonically as τ tightens ($0.249 \rightarrow 0.338$) while hit rate falls, and same-model precision is $\approx 2\times$ cross-model (0.443 vs. 0.223) — a production deployment caching a single model’s responses sits in the most favorable stratum.

6.9 Fairness: two dedicated baselines beaten, one own term nulled

Against FIRM-style control, PolyForge wins Jain’s index 0.9689 vs. 0.9294 in the v1 matrix ($d_z=1.0$, $p = 4.5 \times 10^{-47}$), replicated in v2 (0.9672 vs. 0.929, $d_z=0.95$, $p = 7.5 \times 10^{-44}$). Against the tuned VTC-style replica divider [28] under interference injection (PREREG_VTC.md, 200 runs): **HV1 PASS** — the dedicated fair divider loses composite J ($d_z=-1.12$, $p = 3 \times 10^{-19}$); **HV2 PASS** — and it does not buy a fairness win either: PolyForge is *more* fair on Jain itself (0.9705 vs. 0.9599, $p = 1.5 \times 10^{-6}$) at -35% cost and -36% worst-tenant p95. Scope: this is capacity-allocation fairness at the control plane, not token-level service fairness (Section 7.2).

The controller’s own γ -term is a published null: under injected noisy-neighbor interference, the γ -ablation shows no significant contribution even in the whale-only subgroup where the

injection fires (`FAIRNESS_V2.md`). The term stays in the system as a configurable weight; it is dropped from the claims.

6.10 Security: a timing side channel and its defense frontier

A shared semantic cache leaks prompt membership across tenants from response time alone: an attacker distinguishing whether a victim tenant inserted a prompt reaches AUC 0.88 (Figure 6.7; cf. the prompt-cache audit of Prompt-cache audit authors [21] at the API layer). Per-tenant partitioning returns the attacker to chance (AUC 0.50) at 24% latency cost while keeping 76% of hits — strictly dominating response-time padding (never below AUC 0.73 at any cost) and TTL jitter (chance only at $\sim 90\%$ hit loss) on the leakage-vs-benefit frontier (Figure 6.8). This measurement motivated the per-tenant default in the gateway design (Section 3.1).

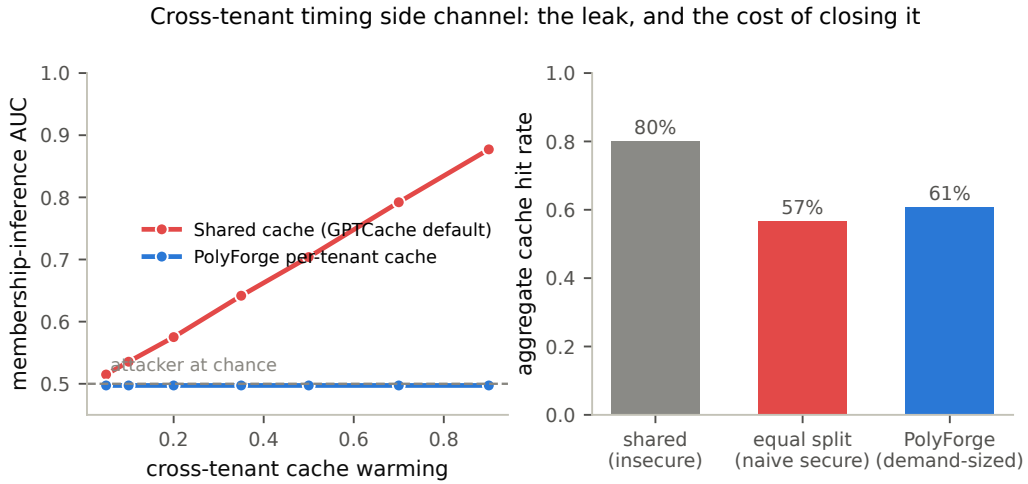


Figure 6.7: Left: membership leakage from response time in a shared semantic cache; per-tenant caching pins the attacker at chance. Right: isolation costs hit rate; demand-proportional sizing recovers part of the penalty.

6.11 Live-cluster validation: ordinal by design

Ground rule: absolutes are never compared across substrates; the live experiment asks only whether the simulator’s *ranking* reproduces under a real scheduler, real HPA, and real load on a kind cluster. Status at the time of writing:

- The **HPA arm is verified live**: a full cluster-backend run valid on a real kind cluster, with eval-export metrics matching the load’s construction exactly.
- The **JCAC arm is fully wired and locally tested** (Section 4.4), gated behind the executable actuation check: a run in which the operator never actuated fails before load is generated.
- The **analysis protocol is frozen** before any live JCAC number exists (`phase7_ordinal.py`, committed with the sim reference slice, 12/12 valid): per-cell winner agreement on

Defense frontier: partitioning dominates the known mitigations

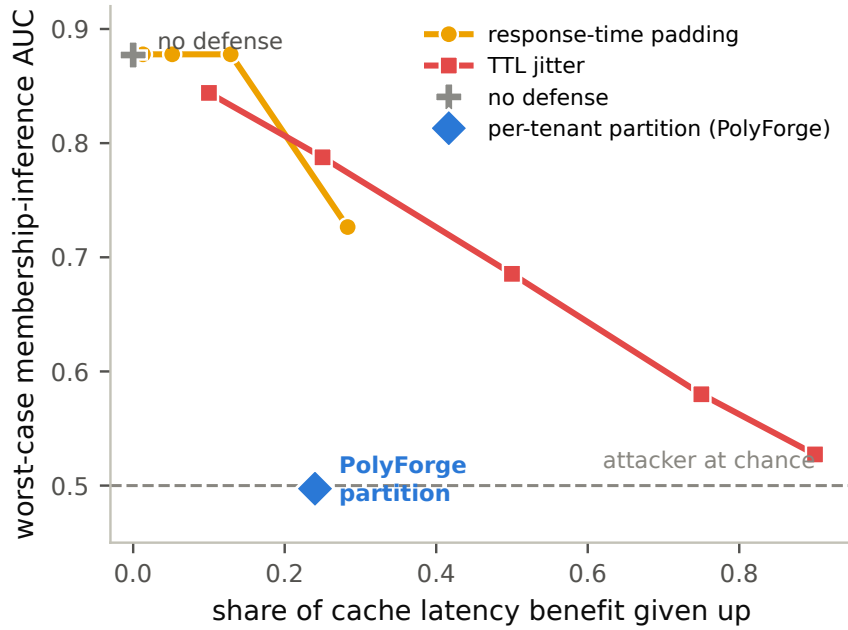


Figure 6.8: Defense frontier: worst-case membership-inference AUC vs. share of the cache’s latency benefit given up. Per-tenant partitioning dominates the frontier rather than merely joining it.

$J/\text{cost}/\text{violation}$, no absolutes across substrates, no significance testing on three repetitions.

The figure caption is frozen verbatim in PHASE7_JCAC_PLAN.md and carries the disclosure that the live data plane burns fixed CPU per request kind, so the cache-size and model-tier knobs are inert live: the live figure validates the *replica-control projection* of the joint controller, while the cache knob’s realism is carried by the LMSYS protocol (Section 6.8) and the tier knob’s by the GPU tier bench (Section 5.3). The live JCAC slice is the one remaining measurement in the project; until it runs, the HPA arm is the only live-verified system, and this thesis says so rather than extrapolating.

6.12 Summary: which number to cite

Claim	Cite	Not
Cost, real demand	-70% ($n=96$, $p \leq 4.3 \times 10^{-6}$)	-76% ($n=16$ point estimate, HT FAIL)
Cost, synthetic matrix	$-36 \dots -48\%$ HPA/KEDA/FIRM	vs. any mixing with replay numbers
Forecasting gain	Holt -26.9% RMSE, real trace	seasonal -44.8% (synthetic; did not transfer)
Forecasting, external	winner tracks measured periodicity (3 traces)	“Holt is best” (scoped to BurstGPT)
SLO	parity at $-43 \dots -48\%$ cost; mechanism $p = 6 \times 10^{-4}$	any “beats HPA/KEDA on violation”
Fairness	Jain 0.9705 vs. VTC 0.9599; FIRM $d_z=1.0$	the γ -ablation (published null)
Cache hit rate	29.6% @ 0.85, real LMSYS, $+93\%$ vs. 10%-fixed	synthetic saturation; the 200-entry strawman
Cache quality	precision-vs- τ shape + strata	“69% of hits are wrong” (ignores the stochasticity ceiling)
Security	AUC $0.88 \rightarrow 0.50$ at 24% latency, 76% hits kept	—
Effect sizes	d_z with 95% bootstrap CI	bare p-values below $\sim 10^{-6}$
Planner scale	exponent 1.45; 3 s timeout crossed at 128 tenants	“linear in tenants”

Table 6.2: Disambiguation table (from RESULTS_MASTER.md); the conservative column is the thesis’s claim.

Chapter 7

Discussion

This chapter confronts the objections a competent examiner should raise — each stated in its strongest form, answered with committed evidence, and never softened. The full pre-answered set is maintained in `docs/DEFENSE_QA.md`; the load-bearing ones follow.

7.1 Threats to validity

“Your headline numbers come from your own simulator.” True, and framed that way everywhere: decision quality under a stated system model. Three things bound the gap. First, *calibration direction*: the congestion form was fitted against a real inference server and the model over-penalizes high utilization ($a=0.86$ vs. assumed 1.0) — it errs against PolyForge’s lean postures; the GPU tier bench confirms tier ordering on real hardware. Second, *real data*: the cost and forecasting claims are restated on replayed BurstGPT demand ($n=96$ confirmatory) and the cache claims are measured on real LMSYS prompts. Third, the *live ordinal check*: the kind-cluster experiment asks only whether the simulator’s ranking reproduces, with the HPA arm verified and the JCAC arm gated behind an executable actuation check. What is not claimed: absolute dollars or latencies transferring to any production fleet.

“You designed the v3 overload cells so your system would win.” The cells were sized by arithmetic on committed model constants, pinned by unit tests, not tuned by trial runs. Three protections: the pre-registration named the sole treatment and hypotheses before any run; a same-envelope control cell (`ramp_gentle`) exists where the alleged advantage should not appear — and it does not; and the confirmatory H1’ still *failed*. A designed regime with an honest null inside it is evidence of measurement, not of rigging.

“ $p = 10^{-33}$? Really?” At hundreds of paired seeded simulator runs, tiny p-values measure the simulator’s determinism, not the effect. The citation unit is the paired effect size with a bootstrap CI (Section 5.4); p-values appear once, as prereg gate outcomes.

“Your SLO story moved the goalposts.” The ledger says otherwise, and the ledger is the answer: seven pre-registrations pushed before their runs, two confirmatory SLO failures published as failures, one mechanism confirmation, and a stopping rule forbidding a third attempt. The final SLO claim — parity at $-43 \dots -48\%$ cost — is what survived, not what

was hoped for. Goalpost-moving is invisible when it happens in private; here it happened in public, in the git history, where it can be diffed.

“Were the baselines actually tuned, or strawmen?” Grid-searched on the paper’s own composite objective, per baseline, sweeps committed. The grids are bounded at a vendor-sane envelope because J improves monotonically toward over-provisioning — an unbounded grid would tune every baseline into a cost blowout and *flatter* PolyForge. Iso-cost variants remove the “it just spends less” objection, and their gate is reported FAIL-honest where it fails.

7.2 Construct scope

Fairness altitude. Jain’s index over SLO satisfaction is not fairness as OSDI defines it: VTC and successors define *service* fairness at token granularity inside a continuous-batching engine. PolyForge’s fairness is capacity allocation across tenants at the control plane, the claims are scoped to that altitude, and the `vtc_replica` baseline is a replica-level transplant of VTC’s least-service-first idea, stated as such. Beating it on Jain says PolyForge allocates capacity more evenly at lower cost — not that it schedules tokens more fairly. Token-level fair schedulers would run *inside* each replica pool PolyForge sizes; the two are complementary.

Two economies in one cost model. Replica-hours are priced as infrastructure (\$0.048/replica-hr; the live arm meters real replica-seconds) while model-tier calls are priced at market API ratios (1:10:100). A self-hosting operator should read the tier ratios as relative GPU-time costs — the tier bench measured 1:1.5:16.6 latency ratios on real hardware: same ordering, compressed scale, disclosed. Spot pricing, MIG partitioning, and fractional GPUs are below the model’s altitude and named out of scope. Reconfiguration friction is not ignored: the realism ablation bills startup lag and cold caches; multi-minute large-model pulls exceed what it models, and the limitation says so.

Cache hits that are wrong answers. Anticipated and measured (Section 6.8): precision with strata and a per-1k-incorrect-hits figure next to the savings, plus the stochasticity ceiling that bounds what the proxy can say. No “quality-adjusted dollar” is invented, because pricing a wrong answer is application-specific. Staleness and TTL are out of scope and listed as such.

7.3 Limitations

- **Substrate.** Headline numbers are simulator decision quality; live validation is ordinal-only, currently verified for the HPA arm with the JCAC live slice pending (Section 6.11).
- **The live figure exercises one knob.** The live data plane burns fixed CPU per request kind, so cache and tier are inert there; the live check validates the replica-control projection, with the other knobs’ realism carried by their own real-data protocols. The frozen caption states this.
- **Latency is p95, request-level.** The simulator’s tail beyond p95 is not credible and is not reported; token-level dynamics (TTFT, decode-length tails, head-of-line

blocking under continuous batching) are outside the model. The live path can report any percentile; p99 live restatement is future work, not silently substituted.

- **Trace scope.** Demand claims rest on BurstGPT plus the Azure LLM 2024 release; prompt claims on LMSYS-Chat-1M. One generalization failure is already published (the seasonal non-transfer), which is the strongest available evidence that the remaining claims are scoped honestly. FaaS/VM traces were deliberately descoped as non-LLM demand.
- **Tenant count.** Eight tenants per cluster is the studied composition factor; the planner’s measured scaling (exponent 1.45, timeout crossover at 128 tenants) bounds, but does not remove, the gap to production tenant populations. Coordination costs outside the planner are unmeasured.
- **Production scale.** SageServe-class fleets serve more in a day than the entire replay; the 10.63M-request demand replay is the honest analog at decision level, and no stronger claim is made.

7.4 The honest-nulls ledger as a contribution

Seven published negatives travel with the wins: v2 H1, v3 H1’, the underpowered first replay, the γ -term null, the seasonal non-transfer, the structurally impossible raw per-metric gate, and the cache hit-quality result. None was hidden; several redirected the design (Holt as recommended default; per-stream forecasting; per-tenant caches). The methodological claim of this thesis — pre-registration with published nulls in a systems evaluation — is independent of any performance number and, in the surveyed literature, unique to this work.

Chapter 8

Conclusion

8.1 What was established

This thesis set out to test whether per-layer local optima compose in multi-tenant hybrid serving, and answered with a built system and a pre-registered measurement campaign. The answer: they do not compose, and the gap is large. A controller that plans replicas, semantic-cache budget, and model tier together beats every tuned per-layer baseline on the composite objective in every campaign that tested it, is the only Pareto-undominated system in a 1,800-run matrix, and on replayed real demand delivers -70% cost against tuned reactive scalers with a disclosed $+0.07$ violation trade. Removing jointness alone costs $+2884\%$ on the cost term — the composition gap made into a number.

Around that central result: an honest SLO account (parity at $-43\ldots-48\%$ cost, a confirmed forecasting mechanism, two published confirmatory failures, and a stopping rule); real-data cache results with a quality denominator no surveyed cache paper publishes; fairness wins over two dedicated baselines at the stated altitude with the controller’s own fairness term honestly nulled; a security frontier showing per-tenant cache partitioning dominates padding and jitter defenses against a demonstrated AUC-0.88 timing side channel; two construction-level guarantees on the deployed solver; and a working Kubernetes-native implementation held to CI gates throughout.

The methodological result may outlast the numbers: seven pre-registrations pushed before their runs, immutable closed campaigns, effect sizes over p-values, substrate separation, and a published honest-nulls ledger — demonstrated in a systems evaluation at thesis scale, in public git history.

8.2 Future work

- **The live JCAC slice.** The one remaining measurement: the jcac smoke and full ordinal runs on a Docker-capable machine, under the already-frozen protocol and caption (PHASE7_JCAC_PLAN.md). Everything is wired and gated; the result will be reported ordinal-only whichever way it lands.
- **Token-level demand signals.** Swap the request-rate DemandSource for TTFT/queue-depth/KV-pressure signals from an llm-d-class engine, keeping the planner unchanged — the designed migration path onto the 2026 substrate (Section 2.4).

- **p99 live restatement.** The live path measures wall-time latency; extending the ordinal protocol to p99 removes the simulator’s p95 scope limit.
- **Planner scaling past 128 tenants.** Planning cells, incremental fairness partial sums, and a faster inner loop are the named levers against the measured 1.45 growth exponent.
- **Adopting the empirical cache curve.** Feeding the measured $h(K)$ fit into the model constants is a new pre-registered experiment by the project’s own rules — deliberately left for one.
- **Prefix/KV-cache layers and token-level fairness.** Both sit below PolyForge’s altitude and compose with it; a joint portfolio-plus-engine study is the natural next thesis.

8.3 Closing remark

The strongest sentence this project can write is not a percentage. It is that every percentage in it can be traced — to a protocol committed before the run, a script that regenerates it, and a git history that would have exposed any other way of obtaining it. That standard is portable to any systems evaluation, and it is the practice this thesis most hopes to see reused.

Appendix A

Claims Traceability

Every claim in this thesis maps to a committed, machine-generated measurement record in the public repository, regenerable by a named script from archived run databases. The consolidated entry point is `research/analysis/RESULTS_MASTER.md`; per-campaign files are immutable once their campaign closes. Paths below are repository-relative; all measurement records live under `research/analysis/` unless prefixed.

Table A.1: Claim-to-evidence map. Raw run databases (DuckDB) are archived under `eval/results/`; the Zenodo deposit bundles them with a manifest (`docs/RELEASE_CHECKLIST.md`).

Claim	Evidence record	Protocol / regeneration
Wins composite J and cost vs. all five tuned baselines, 1,800-run matrix; raw per-metric gate FAIL	RESULTS.md	v1 gate frozen in roadmap; <code>run_analysis.py</code>
Ablations: $-$ joint +2884% cost; $-$ eviction +35.7%; $-$ classifier $-$ 45% relative hits	RESULTS.md (ablation tables)	<code>run_analysis.py</code>
Holt $-$ 16.3% violation in-loop; realism ablation; self-calibration $d_z=+0.46$; side channel AUC 0.88 $\rightarrow$ 0.50	ADVANCED.md, figs. 13–17	<code>advanced.py</code>
v2: H1 FAIL (parity at $-43\ldots-48\%$ cost), H2 PASS (-11% vs. v1); iso-cost gate outcomes	RESULTS_V2.md	PREREG_V2.md; <code>analysis_v2.py</code>
v3: H1' FAIL ($3.2\text{--}3.4\times$ spend), H2' PASS ($p = 6\times 10^{-4}$); spike-class exploratory win	RESULTS_V3.md	PREREG_V3.md (pushed pre-run); <code>analysis_v3.py</code>
Replay $n=16$: HT FAIL, point estimates reproduce v1 ranking	RESULTS_TRACE.md	PREREG_TRACE.md; <code>trace_matrix.py</code>
Replay $n=96$: HT2 PASS, cost -70% , violation $+0.07$ disclosed	RESULTS_TRACE2.md	PREREG_TRACE2.md; <code>trace_matrix2.py</code>
Effect sizes with 95% bootstrap CIs (citable unit)	EFFECT_SIZES.md	<code>effect_sizes.py</code>

Table A.1: Claim-to-evidence map. Raw run databases (DuckDB) are archived under `eval/results/`; the Zenodo deposit bundles them with a manifest (`docs/RELEASE_CHECKLIST.md`).

Claim	Evidence record	Protocol / regeneration
BurstGPT forecast ablation: Holt -26.9% (seg. 1); seasonal non-transfer	FORECAST_TRACE_REAL.md	forecast_trace_real.py; ETL research/traces/ fetch_burstgpt.py
Azure LLM 2024: mechanism boundary reproduces; pooling destroys forecastability	FORECAST_AZURE.md	protocol commit 573c197 + declared amendment 399f86a; forecast_azure.py
LMSYS cache headline: 29.6% @ 0.85 ; $+93\%$ vs. 10% -fixed; $\$0.296/1k$; $h(K)$ fit	SEMANTIC_CACHE.md	pre-download amend- ment 31c73c7; semantic_cache_eval.py
Cache hit precision: 0.313 @ $\tau=0.85$; strata; stochasticity ceiling 0.338	CACHE_PRECISION.md	protocol flb2988 (pushed pre-run); cache_hit_precision.py
FIRM fairness win: Jain 0.9689 vs. 0.9294 , $d_z=1.0$ (v1); replicated $d_z=0.95$ (v2)	RESULTS.md, RESULTS_V2.md	run_analysis.py, analysis_v2.py
VTC-replica: HV1+HV2 PASS; Jain 0.9705 vs. 0.9599 at -35% cost	VTC_FAIRNESS.md	PREREG_VTC.md; analysis_vtc.py
γ -term null under injected interference	FAIRNESS_V2.md	fairness_v2.py
Objective-weight sensitivity: direction never flips in 425 cells	SENSITIVITY_J.md	sensitivity_j.py (de- clared before run)
Solver propositions (block-coordinate optimality; calibration bounds)	THEORY_V2.md	proofs against research/ jcac_sim/controller.py
Congestion calibration $a=0.86$ (errs against us); GPU tier ordering 1:1.5:16.6	research/calibration/ CALIBRATION.md, TIER_BENCH.md	calibration scripts; kaggle_tier_bench.py
Planner scaling: exponent 1.45 ; timeout crossover at 128 tenants	PLANNER_SCALING.md	planner_scaling.py
Live HPA arm verified; JCAC arm wired + gated; ordinal protocol frozen pre-run	docs/SESSION_LOG.md (16d, 17), docs/PHASE7_JCAC_PLAN.md	scripts/ phase7_kind_run.sh; phase7_ordinal.py (com- mit 650ce29)
Baseline tuning grids and bounds	eval/baselines/TUNING.md	grids in eval/baselines/grids/

Reading order for an examiner. (1) RESULTS_MASTER.md — scoreboard, ledger, disambiguation; (2) the pre-registrations, in commit order, against their result files;

(3) docs/DEFENSE_QA.md — thirteen hard questions pre-answered; (4) docs/SESSION_LOG.md — the session-by-session verification record.

Bibliography

- [1] AIBrix contributors. AIBrix: Cost-efficient and scalable LLM inference infrastructure, 2025. <https://github.com/vllm-project/aibrix>.
- [2] Aladdin authors. Aladdin: Joint placement and scaling for SLO-aware LLM serving, 2024. arXiv:2405.06856, <https://arxiv.org/abs/2405.06856>.
- [3] Fu Bang. GPTCache: An open-source semantic cache for LLM applications. In *Proceedings of NLP-OSS*, 2023. <https://aclanthology.org/2023.nlposs-1.24/>.
- [4] BurstGPT authors. BurstGPT: A real-world workload dataset to optimize LLM serving systems, 2024. arXiv:2401.17644, <https://arxiv.org/abs/2401.17644>.
- [5] Chiron authors. Chiron: Hierarchical autoscaling for large language model serving, 2025. arXiv:2501.08090, <https://arxiv.org/abs/2501.08090>.
- [6] D2LPM authors. Locality-aware fair scheduling for LLM serving, 2025. arXiv:2501.14312, <https://arxiv.org/abs/2501.14312>.
- [7] Equinox authors. Equinox: Holistic fair scheduling for LLM serving, 2025. arXiv:2508.16646, <https://arxiv.org/abs/2508.16646>.
- [8] Faro authors. Faro: SLO-aware serving on on-premise inference clusters, 2024. arXiv:2409.19488, <https://arxiv.org/abs/2409.19488>.
- [9] Charles C. Holt. Forecasting seasonals and trends by exponentially weighted moving averages. *International Journal of Forecasting*, 20(1):5–10, 2004.
- [10] InstCache authors. InstCache: A predictive instruction cache for LLM serving, 2024. arXiv:2411.13820, <https://arxiv.org/abs/2411.13820>.
- [11] Rajendra K. Jain, Dah-Ming W. Chiu, and William R. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report DEC-TR-301, Digital Equipment Corporation, 1984.
- [12] Joint allocation authors. Joint resource allocation for SLO-constrained LLM inference, 2026. arXiv:2604.07472, <https://arxiv.org/abs/2604.07472>.
- [13] Justitia authors. Justitia: Fair and efficient scheduling for LLM serving, 2025. arXiv:2510.17015, <https://arxiv.org/abs/2510.17015>.
- [14] KEDA contributors. KEDA: Kubernetes event-driven autoscaling, 2025. <https://keda.sh>.

- [15] Kubernetes contributors. Horizontal pod autoscaling, 2025. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.
- [16] Kubernetes WG-Serving. Gateway API inference extension, 2025. <https://github.com/kubernetes-sigs/gateway-api-inference-extension>.
- [17] llm-d contributors. llm-d: Kubernetes-native distributed LLM inference, 2025. <https://github.com/llm-d/llm-d>.
- [18] MeanCache authors. MeanCache: User-centric semantic caching for LLM web services, 2024. arXiv:2403.02694, <https://arxiv.org/abs/2403.02694>.
- [19] Microsoft Azure. Azure LLM inference traces 2024 (AzurePublicDataset), 2024. <https://github.com/Azure/AzurePublicDataset>.
- [20] NVIDIA. NVIDIA dynamo: Disaggregated prefill/decode serving, 2025. <https://github.com/ai-dynamo/dynamo>.
- [21] Prompt-cache audit authors. Auditing prompt caching in language model APIs, 2025. arXiv:2502.07776, <https://arxiv.org/abs/2502.07776>.
- [22] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020.
- [23] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [24] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. INFaaS: Model-less inference serving, 2019. arXiv:1905.13348, <https://arxiv.org/abs/1905.13348>.
- [25] SageServe authors. Sageserve: Forecast-aware auto-scaling for LLM serving at scale. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (PO-MACS/SIGMETRICS)*, 2025. <https://dl.acm.org/doi/10.1145/3771576>, <https://arxiv.org/abs/2502.14617>.
- [26] SCALM authors. SCALM: Semantic caching for large language model services, 2024. arXiv:2406.00025, <https://arxiv.org/abs/2406.00025>.
- [27] Semantic caching adaptation authors. Semantic caching: From offline evaluation to online adaptation, 2025. arXiv:2508.07675, <https://arxiv.org/abs/2508.07675>.
- [28] Ying Sheng et al. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, 2024. <https://arxiv.org/abs/2401.00588>.
- [29] SLO-driven autoscaling authors. SLO-driven, cost-aware autoscaling for kubernetes inference services, 2025. arXiv:2512.23415, <https://arxiv.org/abs/2512.23415>.

- [30] Tale of Two Scales authors. A tale of two scales: Reconciling horizontal and vertical scaling for inference serving, 2024. arXiv:2407.14843, <https://arxiv.org/abs/2407.14843>.
- [31] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [32] Lianmin Zheng et al. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)*, 2024.